CDC-CNPJ — Dossiê Técnico para a Equipe

Fontes de dados, plano de execução e problemas de engenharia

Versão 1.0 — Agosto/2026

PARTE 1 — AS FONTES EM DETALHE

1.1 Fonte primária: Dados Abertos do CNPJ (Receita Federal)

Não é uma API. É a ausência de uma API — e é exatamente por isso que o projeto existe.

Atributo	Detalhe
URL oficial	`https://arquivos.receitafederal.gov.br` (índice de arquivos)
Layout oficial	`https://www.gov.br/receitafederal/dados/cnpj-metadados.pdf`
Catálogo	dados.gov.br → "Cadastro Nacional da Pessoa Jurídica - CNPJ"
Autenticação	Nenhuma
Frequência	Mensal (pastas por competência YYYY-MM)
Formato	ZIPs contendo CSVs com separador `;`, encoding legado (atenção: não é UTF-8 puro — validar na Fase 1)
Volume	Dezenas de GB descompactado (~85 GB relatados); compactado na casa de poucos GB por família de arquivos
Licença	Dados abertos, uso comercial livre

Estrutura dos arquivos por competência mensal:

Família	Conteúdo	Chave	Observação
`Empresas*.zip` (múltiplas partes)	Razão social, natureza jurídica, capital social, porte, qualificação do responsável	`cnpj_basico` (8 posições)	Nível "matriz lógica" da empresa
`Estabelecimentos*.zip` (múltiplas partes — **o grosso do volume**)	CNPJ completo, nome fantasia, situação cadastral + data + motivo, endereço completo (tipo logradouro, logradouro, número, bairro, CEP, município, UF), CNAE principal e secundários, telefones, e-mail, datas	CNPJ completo (básico+ordem+DV)	**A tabela mais valiosa: endereço + situação + CNAE**
`Socios*.zip` (múltiplas partes)	Identificador do sócio, nome, CPF mascarado (`***XXXXXX**`), qualificação, faixa etária, data de entrada, representante legal	`cnpj_basico` + identificador	**Não há chave primária estável do sócio** — problema central (ver Parte 3)
`Simples.zip`	Opção pelo Simples/MEI, datas de opção e exclusão	`cnpj_basico`	Arquivo único
`Cnaes.zip`, `Motivos.zip`, `Municipios.zip`, `Paises.zip`, `Naturezas.zip`, `Qualificacoes.zip`	Tabelas de domínio (código → descrição)	código	Pequenas; mudam raramente, mas mudam
`entidades-lucro-*.zip`, `entidades-imunes-e-isentas.zip`	Regime tributário	cnpj	Distribuídos à parte; layout próprio

Fatos operacionais que a equipe precisa saber:

1. **Servidores lentos.** Download direto da Receita é notoriamente instável e demorado. Plano A: espelho da **Casa dos Dados** (`dados-abertos-rf-cnpj.casadosdados.com.br`), que replica mensalmente os arquivos oficiais atrás de CDN Cloudflare. Plano B:

origem oficial. Sempre registrar hash (SHA-256) e origem de cada arquivo baixado — reprodutibilidade é requisito do TCC.

2. **Arquivos particionados sem garantia de ordenação.** `Estabelecimentos0.zip`, `Estabelecimentos1.zip`... Não assumir que o mesmo CNPJ cai na mesma parte em meses diferentes. O diff é sobre o conjunto lógico, nunca arquivo-a-arquivo.
3. **A tabela de municípios usa código próprio da Receita/SIAFI, não o código IBGE de 7 dígitos.** Se quisermos interoperar com o resto do ecossistema (e queremos — é a chave universal), precisamos da tabela de correspondência (o Tesouro Nacional publica `tabmun.csv`; o Minha Receita usa exatamente essa ponte). Isso vira um passo da normalização.
4. **CNPJ alfanumérico (desde 31/07/2026).** Novas inscrições podem ter letras nas 8 primeiras posições. Consequências diretas: nada de armazenar CNPJ como `long`/`BIGINT`; ordenação passa a ser lexicográfica; o dígito verificador tem cálculo próprio para o formato misto; particionamento por faixa numérica quebra — usar hash da string. Somos possivelmente o primeiro pipeline aberto nascido já compatível. Isso é seção de destaque na monografia.

1.2 Fontes de apoio e validação

IBGE — API de Localidades (`servicodados.ibge.gov.br/api/docs`) REST aberto, sem autenticação, JSON. Uso no projeto: malha oficial de municípios (código de 7 dígitos) para enriquecer eventos com a chave geográfica universal, via ponte com a tabela da Receita. Também fornece a estrutura CNAE oficial para validar códigos.

Consulta pública unitária de CNPJ (site da Receita) Não é API formal, mas é o nosso **oráculo de corretude**: para validar amostras do diff ("o evento diz que a empresa X foi baixada em maio — a consulta oficial confirma?"). Uso manual/amostral apenas, respeitando limites do serviço. Alternativa programática amostral: API do projeto Minha Receita (`minhareceita.org`), aberta, que serve o mesmo dado por CNPJ.

dados.gov.br (CKAN API) Metacatálogo. Uso pontual: monitorar mudanças na página do conjunto CNPJ (avisos de layout novo, mudança de endereço dos arquivos).

1.3 O que estudar como referência de engenharia (não como dependência)

- **minha-receita (Cuducos)** — ETL Go maduro sobre os mesmos arquivos; documentação do ETL descreve armadilhas reais do parsing (campos com aspas, encoding, a ponte de municípios). Ler o código antes de escrever o nosso parser economiza semanas.
- **libercapital/dados_publicos_cnpj_receita_federal** — mantém **testes agendados semanalmente para detectar mudança de formatação nos arquivos**. Roubar essa ideia: nosso pipeline terá um estágio de validação de schema que falha ruidosamente se a Receita mudar algo.
- **okfn-brasil/receita** — documenta as regras de negócio do identificador (básico 8 + ordem 4 + DV 2) e a consolidação empresa×estabelecimento×sócio.

- **Debezium (docs/design)** — vocabulário e semântica de eventos CDC (create/update/delete, envelope before/after). Vamos falar a língua que o mercado já entende.

PARTE 2 — COMO FAREMOS ISSO DAR CERTO (plano de execução)

2.1 Princípio norteador

Snapshot é imutável; mudança é evento. Todo o sistema deriva disso: snapshots canônicos versionados são a fonte da verdade; o diff é uma função pura entre dois snapshots; os eventos são o produto. Se qualquer coisa der errado, reprocessamos — nada se perde, tudo é reprodutível. (É a mesma filosofia de event sourcing que vocês já conhecem, aplicada de fora para dentro.)

2.2 Pipeline em cinco estágios

```
[1 INGESTÃO]      download (espelho→oficial) + SHA-256 + manifesto da
competência
      ▼
[2 NORMALIZAÇÃO]  CSV posicional/; + encoding legado  →  Parquet canônico
                  • schema explícito e versionado
                  • CNPJ como STRING (alfanumérico-safe)
                  • ponte município Receita→IBGE
                  • particionado por hash(cnpj_basico) em N buckets fixos
      ▼
[3 DIFF]          f(snapshot M-1, snapshot M) → conjunto de mudanças
                  três motores plugáveis atrás da mesma interface:
                  E1 Postgres FULL OUTER JOIN (baseline/corretude)
                  E2 sort-merge externo (memória constante)
                  E3 hash por registro + comparação paralela por bucket
      ▼
[4 EVENTOS]       mudanças → envelope de evento tipado (JSON Schema/Avro
versionado)
                  publicação em Kafka, key = cnpj_basico, tópicos por
entidade
      ▼
[5 CONSUMO DEMO]  consumer que materializa um "changelog consultável"
                  (prova de utilidade + ferramenta de QA do próprio time)
```

Decisão de arquitetura importante — particionamento estável no estágio 2: ao gravar o Parquet canônico já particionado por `hash(cnpj_basico) % N` (N fixo, ex.: 256), garantimos que **o mesmo CNPJ cai no mesmo bucket em todos os meses.** O diff E3 vira então N comparações independentes bucket-a-bucket, paralelizáveis e com memória limitada por bucket. É o truque que faz 85 GB caberem numa máquina comum.

2.3 Contrato de eventos (rascunho para discutir)

Tópicos: `cnpj.empresas`, `cnpj.estabelecimentos`, `cnpj.socios`, `cnpj.simples`, `cnpj.dominio`.

Envelope (inspirado em Debezium):

```json
{
  "schema_version": "1.0",
  "event_id": "uuid-v7",
  "event_type": "estabelecimento.situacao_alterada",
  "entity_key": { "cnpj": "12ABC34500019X" },
  "competencia_de": "2026-06",
  "competencia_para": "2026-07",
  "before": { "situacao_cadastral": "02", "data_situacao": "2019-03-11" },
  "after":  { "situacao_cadastral": "08", "data_situacao": "2026-07-02" },
  "detected_at": "2026-08-15T04:12:03Z",
  "pipeline": { "estrategia": "E3", "run_id": "..." }
}
```

Taxonomia inicial: `empresa.criada|encerrada|alterada`, `estabelecimento.criado|situacao_alterada|endereco_alterado|cnae_alterado|alterado`, `socio.incluido|excluido|alterado`, `simples.opcao_alterada`, `dominio.tabela_alterada`. Eventos "alterado" genéricos carregam o conjunto de campos que mudou — consumidores filtram.

Regra de ouro: **um evento representa a diferença entre duas competências, não uma data exata de mudança.** A granularidade temporal do dado de origem é mensal; nosso contrato é honesto sobre isso (o campo `competencia_de/para` existe por essa razão).

2.4 Sequência de trabalho (o caminho crítico)

Semanas 1–3 — "Um bucket de ponta a ponta". Baixar 2 competências, normalizar SÓ 1 bucket (1/256 do dado, ~330 MB), rodar E1 nesse bucket, emitir eventos em Kafka local via Docker Compose. Objetivo: pipeline inteiro funcionando em miniatura antes de qualquer otimização. Isso mata o risco de integração cedo.

Semanas 4–6 — Caracterização completa (Fase 1 do TCC). Normalização das competências inteiras; medir: nº de registros por tabela, taxa de mudança mensal real por tipo (quantas aberturas? quantas mudanças de endereço?), distribuição de tamanho por bucket. Esses números calibram tudo que vem depois e viram a primeira seção de resultados da monografia.

Semanas 7–14 — E2 e E3 + contrato de eventos v1.0. Implementar as duas estratégias escaláveis; congelar o schema de eventos; validação cruzada tripla (E1≡E2≡E3 no mesmo par de snapshots — qualquer divergência é bug de alguém).

Semanas 15–20 — Bancada experimental. Ambiente fixo documentado, execuções repetidas, coleta de métricas (tempo, memória, I/O, throughput Kafka, custo estimado em instância equivalente). Validação amostral contra a consulta oficial (ex.: 200 eventos sorteados, conferência manual/Minha Receita).

Semanas 21+ — Análise, escrita, consumer demo polido.

Divisão natural em equipe: (A) ingestão+normalização, (B) motores de diff, (C) Kafka+contrato+consumer, (D) bancada experimental+análise. As interfaces entre estágios são arquivos Parquet e tópicos Kafka — acoplamento mínimo, dá pra trabalhar em paralelo desde a semana 1.

2.5 Stack

Java/Kotlin 21 + Spring Boot (serviços de ingestão e consumer) · processamento de diff em Kotlin puro ou com Apache Spark local como comparativo extra (decidir na Fase 1 — Spark entra só se agregar ao experimento, não por moda) · Apache Kafka + Schema Registry (Avro) · PostgreSQL (E1 e consumer demo) · Parquet (camada canônica) · Docker Compose (dev) · métricas via Micrometer + medições de sistema.

PARTE 3 — OS PROBLEMAS QUE VAMOS RESOLVER

Estes são os problemas reais de engenharia/pesquisa. Cada um é uma seção de "desafios e soluções" na monografia — e é neles que a banca vai focar.

P1. Diff de dezenas de GB sem acesso à origem (o problema-título)

CDC clássico usa o log de transações do banco. Nós não temos banco — temos dois arquivos gigantes. A literatura chama isso de snapshot differencing e o trata como último recurso; nós vamos tratá-lo como primeira classe: comparar formalmente três estratégias e publicar em que escala cada uma domina. **Contribuição: os números que hoje só existem dentro de caixas-pretas comerciais.**

P2. Identidade de registro sem chave primária confiável (o problema mais difícil)

Empresas e estabelecimentos têm chave natural (CNPJ). **Sócios não têm**: o mesmo sócio entre dois meses é "o mesmo" por combinação de CNPJ + nome + CPF mascarado + qualificação — e qualquer um desses campos pode mudar. Se errarmos o pareamento, geramos falsos `socio.excluido`+`socio.incluido` em vez de `socio.alterado`. Solução proposta: chave composta hierárquica com regras de desempate documentadas + métrica de "eventos fantasma" na avaliação. É um problema clássico de record linkage aplicado — rende discussão rica na defesa.

P3. Delete vs. desaparecimento

Registro presente em M-1 e ausente em M: foi excluído de verdade ou é falha/reprocessamento do arquivo da Receita? Emitir `*.excluido` ingenuamente pode poluir o stream com falsos negativos da fonte. Solução: quarentena — desaparecimentos viram evento provisório e só confirmam se persistirem na competência seguinte (ou via checagem amostral no oráculo). Trade-off latência×precisão explicitado e medido.

P4. CNPJ alfanumérico

Todo o legado assume dígito. Nós nascemos string-first: tipos, DV do formato misto, ordenação lexicográfica, particionamento por hash. Problema novo (jul/2026), sem literatura consolidada — nossa documentação do impacto arquitetural é contribuição por si só.

P5. Encoding e parsing de CSV legado em escala

Encoding não-UTF-8, campos com aspas e ponto-e-vírgula embutido, layout posicional, partições múltiplas. Parsing ingênuo corrompe dados silenciosamente — e diff sobre dado corrompido gera eventos falsos. Solução: camada de normalização com schema explícito, contadores de rejeição por arquivo e validação estatística (nº de linhas lidas vs. esperadas). O estágio que falha ruidosamente em vez de mentir baixinho.

P6. Evolução de layout pela fonte

A Receita já mudou o formato mais de uma vez na história (2019, 2021) e quebrou todos os projetos da comunidade. Solução: schema versionado + teste de contrato na ingestão (na linha do que a Liber Capital automatizou) + snapshots do estudo congelados no início (o experimento não depende de meses futuros).

P7. Semântica de entrega e replay no Kafka

Que garantia damos ao consumidor? Proposta: at-least-once + eventos com `event_id` determinístico (derivado de run + entidade + tipo), tornando o consumo idempotente barato; tópicos com retenção longa para permitir replay do histórico de mudanças desde o início do projeto. Discussão clássica de sistemas distribuídos — território nosso.

P8. Corretude sem ground truth completo

Não existe gabarito oficial do "diff verdadeiro". Solução em três camadas: (i) validação cruzada E1≡E2≡E3 (independência de implementação como evidência); (ii) validação amostral contra a consulta oficial/Minha Receita; (iii) invariantes de sanidade (ex.: total de `empresa.criada` − `empresa.encerrada` ≈ variação do total de registros). Metodologia de validação é, ela própria, contribuição.

P9. Custo e reprodutibilidade

Meta declarada (H2): ciclo completo em <24h numa máquina de commodity/instância barata. Se conseguirmos, provamos que a barreira de acesso ao delta do CNPJ é artificial — qualquer pesquisador, jornalista ou órgão de controle pode rodar. Se não conseguirmos, os números explicam exatamente por quê. Nos dois casos, resultado publicável.

Resumo em uma frase para alinhar o time

> Vamos pegar o maior cadastro empresarial aberto do país, que hoje só existe como fotografia mensal de 85 GB, e transformá-lo num changelog de eventos que qualquer sistema consome — medindo cientificamente o custo de cada jeito de fazer isso, e publicando tudo aberto.

Primeiras ações concretas: (1) baixar as competências de junho e julho/2026 pelo espelho e guardar com hash; (2) ler o layout oficial (cnpj-metadados.pdf) e o ETL do minha-receita; (3) subir o Docker Compose com Kafka + Postgres; (4) fatiar o bucket 0 e fazer o primeiro diff de brinquedo. Dá pra ter evento de verdade fluindo no Kafka em duas semanas.